



INSTITUTO NACIONAL SIMÓN BOLÍVAR



MÓDULO 3.3 “Administración de Bases de Datos”

Tarea:
consultas CRUD y JOIN

Docente: Raquel Esperanza Medrano

Opción: 3° año BTV en Desarrollo de Software

Nombre del alumno:

Vanesa Alejandra Amaya	#2
José Enmanuel Ortiz Mejía	#20

Imagen 1 — INSERT de alojamiento

Se usa `INSERT INTO accommodations (...) VALUES (...)`. Básicamente le decimos a la base de datos: "agrega una fila nueva a la tabla de alojamientos con estos datos exactos". Se especifican entre paréntesis los nombres de las columnas (id, dueño, tipo, ubicación, nombre, descripción, etc.) y luego, en el mismo orden, los valores que le corresponden a cada una. Después se usa `SELECT * FROM accommodations` (el asterisco significa "todas las columnas") para traer toda la tabla y comprobar visualmente que el nuevo alojamiento quedó guardado.

The screenshot shows a PostgreSQL client window with the following content:

Consulta

```
1463 '2026-04-17 01:23:25.356206');
1464 SELECT * FROM accommodations
1465
1466 -- 3. Insertar huéspedes y reserva lo iso jose
1467 INSERT INTO booking_guests (booking_guest_id, booking_id, first_name, last_name, age,
1468 document_number, created_at)
1469 VALUES ('104', '87', 'Daniel', 'Pérez',
1470 NULL, NULL, '2026-04-15 01:23:25.356206');
1471 SELECT * FROM booking_guests
1472
1473
1474
1475 -- 4. Insertar pago lo iso jose
1476 INSERT INTO bookings (booking_id, guest_id, accommodation_id, room_id, booking_status_id,
1477 check_in_date, check_out_date, adult_count, child_count, subtotal_amount, tax_amount,
1478 discount_amount, total_amount, special_requests, booking_reference, booked_at, created_at, updated_at)
1479 VALUES ('101', '100', '8', NULL, '4', '2026-03-26', '2026-03-31', '3', '0', '817.49',
```

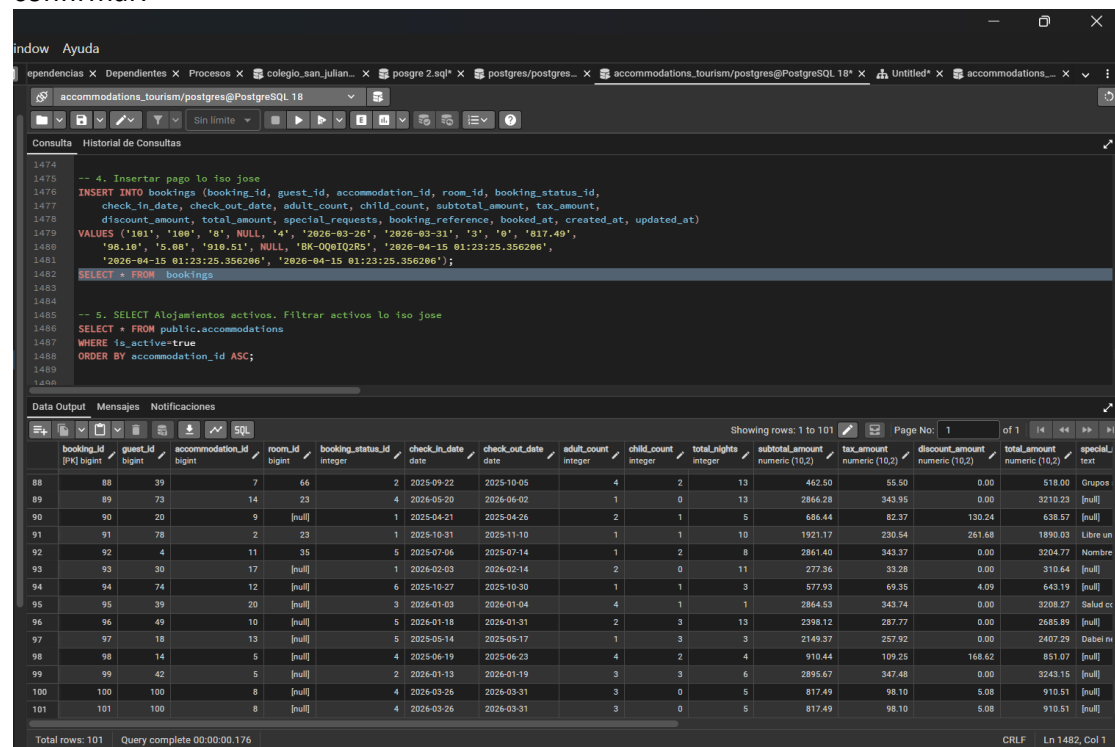
Data Output

booking_guest_id [PK] bigint	booking_id bigint	first_name character varying (100)	last_name character varying (100)	age integer	document_number character varying (50)	created_at timestamp without time zone
90	90	Israel	Linke	[null]	CJ26667890	2026-04-15 01:23:25.356206
91	91	Jennifer	Bohnbach	[null]	[null]	2026-04-15 01:23:25.356206
92	92	Matthew	Leleu	[null]	sA60602908	2026-04-15 01:23:25.356206
93	93	Maggie	Jahn	[null]	Ps50489267	2026-04-15 01:23:25.356206
94	94	Emine	Siering	45	yU56341085	2026-04-15 01:23:25.356206
95	95	Lilia	Hölzenbecher	25	[null]	2026-04-15 01:23:25.356206
96	96	Esteban	Rohleder	18	[null]	2026-04-15 01:23:25.356206
97	97	Azahar	Questa	[null]	HJ08943468	2026-04-15 01:23:25.356206
98	98	Mckenzie	Vera	[null]	vp10174907	2026-04-15 01:23:25.356206
99	99	Carole	Scheibe	[null]	[null]	2026-04-15 01:23:25.356206
100	100	Anel	Luis	60	[null]	2026-04-15 01:23:25.356206
101	101	Odula	Ocasio	21	[null]	2026-04-15 01:23:25.356206
102	102	Jessica	Centú	[null]	[null]	2026-04-15 01:23:25.356206
103	103	Ascensión	Grotmer	[null]	[null]	2026-04-15 01:23:25.356206
104	104	Daniel	Pérez	[null]	[null]	2026-04-15 01:23:25.356206

Total rows: 104 Query complete 00:00:00.215 CRLF Ln 1472, Col 1

Imagen 2 — INSERT de huésped/reserva

Mismo patrón de INSERT INTO, pero ahora sobre la tabla booking_guests. Aquí se nota que algunos campos se dejan como NULL, que en SQL significa "sin valor" — en este caso no se conocía la edad ni el número de documento del huésped, así que se deja vacío en lugar de inventar un dato. Luego, otra vez SELECT * FROM para confirmar.



The screenshot shows a PostgreSQL IDE window with a query editor and a results pane. The query editor contains the following SQL code:

```
-- 4. Insertar pago lo iso jose
INSERT INTO bookings (booking_id, guest_id, accommodation_id, room_id, booking_status_id,
check_in_date, check_out_date, adult_count, child_count, subtotal_amount, tax_amount,
discount_amount, total_amount, special_requests, booking_reference, booked_at, created_at, updated_at)
VALUES ('101', '100', '0', NULL, '4', '2026-03-26', '2026-03-31', '3', '0', '817.49',
'98.10', '5.08', NULL, 'BK-00@IQ2R5', '2026-04-15 01:23:25.356206',
'2026-04-15 01:23:25.356206');

SELECT * FROM bookings;

-- 5. SELECT Alojamientos activos. Filtrar activos lo iso jose
SELECT * FROM public.accommodations
WHERE is_active=true
ORDER BY accommodation_id ASC;
```

The results pane shows the output of the SELECT * FROM bookings query. It displays a table with 18 columns and 101 rows. The columns are: booking_id, guest_id, accommodation_id, room_id, booking_status_id, check_in_date, check_out_date, adult_count, child_count, total_nights, subtotal_amount, tax_amount, discount_amount, total_amount, special_requests, booking_reference, booked_at, and created_at. The first row (101) shows the data for the booking inserted in the query above.

booking_id	guest_id	accommodation_id	room_id	booking_status_id	check_in_date	check_out_date	adult_count	child_count	total_nights	subtotal_amount	tax_amount	discount_amount	total_amount	special_requests	booking_reference	booked_at	created_at
101	100	0	NULL	4	2026-03-26	2026-03-31	3	0	5	817.49	98.10	5.08	910.51	[null]	BK-00@IQ2R5	2026-04-15 01:23:25.356206	2026-04-15 01:23:25.356206

Imagen 3 — INSERT de pago/reserva

Igual estructura de INSERT INTO bookings, pero esta tabla tiene más columnas porque una reserva carga mucha información: fechas de entrada y salida, cantidad de adultos y niños, subtotal, impuestos, descuento, total y hasta un código de referencia único para identificar la reserva. Es el INSERT más largo porque la tabla de reservas es la más "cargada" de datos del sistema.

Window Ayuda

dependencias X Dependientes X Procesos X colegio_san_julian... X posgre 2.sql* X postgres/postgres... X accommodations_tourism/postgres@PostgreSQL 18* X Untitled* X accommodations_...

accommodations_tourism/postgres@PostgreSQL 18

Consulta Historial de Consultas

```

1482 SELECT * FROM bookings
1483
1484
1485 -- 5. SELECT Alojamientos activos. Filtrar activos lo iso jose
1486 SELECT * FROM public.accommodations
1487 WHERE is_active=true
1488 ORDER BY accommodation_id ASC;
1489
1490
1491 -- 6. SELECT Huéspedes por país. Filtrar por nacionalidad lo iso jose
1492 SELECT * FROM public.guests
1493 WHERE nationality = 'Irak'
1494 ORDER BY guest_id ASC;
1495
1496 SELECT * FROM public.guests
1497 WHERE nationality = 'Japón'
1498 ORDER BY guest_id ASC;
1499

```

Data Output Mensajes Notificaciones

Showing rows: 1 to 18 Page No: 1 of 1

accommodation_id [PK] bigint	owner_id bigint	accommodation_type_id integer	location_id bigint	name character varying (150)	description text
5	6	18	3	Rustic Getaway de la Monta...	Couper respect soutenir enfoncer encore.
6	7	17	8	Panoramic Getaway dan	María pregunta hasta siete esos.
7	8	2	4	Cozy Lodge Ville	Et considérer poste fait retomber disparaître. Prononcer avenir observer saint. École bonheur environ écrire hiver printemps combat.
8	9	19	8	Boutique Nest boeuf	Oser habiter passage respecter assez planche.
9	10	2	2	Rustic Lodge boeuf	Denken nimmt später. Darauf Ferien plötzlich.
10	11	15	4	Boutique Stay Ville	Dick kennen heißen Katze Berg dein Haus gerade.
11	12	13	1	Rustic Retreat Ville	Material hit no energy.
12	13	18	8	Panoramic Suite Ville	Acciones términos nuestro están cada. Pp imagen durante acto. Existencia sable resto cabeza cierto pasado. Cuadro militar casa punto.
13	14	17	3	Luxury Escape de la Montaña	Midi falloir revenir mesure or bas.
14	15	19	1	Luxury Nest Ville	Régler saint autrefois vie patron Noël. Étendre soudain renoncer passé confier vers guerre. Sauver lune emmener classe.
15	17	9	3	Luxury Getaway los alhos	Unten Musik trinken Angst aus Pferd. Freude erzählen müde nämlich antworten warten Mensch. Les suchen schlafen geben beißen. Dir ihr Kopf Glück.
16	18	17	1	Charming Stay los bajos	Momento acción cuales tuvo niño mujeres. Nacional mi deseo entrar base. Aspectos corte valores manera.
17	19	11	4	Boutique Suite los bajos	Wagen Leherleie bleiben hinein Mein Feuer Name. Baden legen Wald. Fünf Affe Bush davon Freude hat Hunger.
18	20	9	3	Rustic Retreat furt	Crime network available mean share evidence writer. Budget window hour some fund voice sense current. Husband American although require sound mind c...

Imagen 4 — Filtro con WHERE

Aquí aparece SELECT ... WHERE is_active = true. El WHERE es la cláusula que filtra: en vez de traer todas las filas, solo trae las que cumplen la condición — en este caso, solo los alojamientos que están marcados como activos. Y ORDER BY accommodation_id ASC simplemente ordena el resultado de menor a mayor id, para que sea más fácil de leer.

Window Ayuda

dependencias X Dependientes X Procesos X colegio_san_julian... X posgre 2.sql* X postgres/postgres... X accommodations_tourism/postgres@PostgreSQL 18* X Untitled* X accommodations_...

accommodations_tourism/postgres@PostgreSQL 18

Consulta Historial de Consultas

```

1488 WHERE is_active=true
1489 ORDER BY accommodation_id ASC;
1490
1491
1492 -- 6. SELECT Huéspedes por país. Filtrar por nacionalidad lo iso jose
1493 SELECT * FROM public.guests
1494 WHERE nationality = 'Irak'
1495 ORDER BY guest_id ASC;
1496
1497 SELECT * FROM public.guests
1498 WHERE nationality = 'Japón'
1499 ORDER BY guest_id ASC;
1500
1501 SELECT * FROM public.guests
1502 WHERE nationality = 'Iran'
1503 ORDER BY guest_id ASC;
1504

```

Data Output Mensajes Notificaciones

Showing rows: 1 to 1 Page No: 1 of 1

guest_id [PK] bigint	first_name character varying (100)	last_name character varying (100)	email character varying (150)	phone character varying (30)	date_of_birth date	nationality character varying (100)	passport_number character varying (50)	emergency_contact_name character varying (150)	emergency_contact_phone character varying (30)	created_at timestamp with time zone
1	Daniel	Wheeler	moulinnicole@example...	03 73 83 02 84	2000-10-18	Irak	[null]	Edeltrud Jacobi Jäckel	(836)440-8926x87056	2026-04-15 01:...

Ejecución exitosa. Tiempo de ejecución total de la consulta: 149 msec. 1 filas afectadas.

Total rows: 1 Query complete 00:00:00.149 CRLF Ln 1492, Col 1

Imágenes 5, 6 y 7 — Filtrar por nacionalidad

Se repite tres veces la misma lógica: `SELECT * FROM guests WHERE nationality = 'Irak'` (y luego lo mismo cambiando el país a Japón e Irán). Es una forma de mostrar que el `WHERE` puede compararse contra un texto exacto — aquí se está preguntando "tráeme solo los huéspedes cuyo país sea exactamente este". Como el sistema tiene pocos huéspedes de cada nacionalidad, cada consulta regresa solamente 1 resultado.

The screenshot shows the PostgreSQL IDE interface. The query editor contains the following SQL code:

```
1488 -- 6. SELECT Huéspedes por país. Filtrar por nacionalidad lo iso jose
1489 ORDER BY accommodation_id ASC;
1490
1491 -- 6. SELECT Huéspedes por país. Filtrar por nacionalidad lo iso jose
1492 SELECT * FROM public.guests
1493 WHERE nationality = 'Irak'
1494 ORDER BY guest_id ASC;
1495
1496 SELECT * FROM public.guests
1497 WHERE nationality = 'Japón'
1498 ORDER BY guest_id ASC;
1499
1500 SELECT * FROM public.guests
1501 WHERE nationality = 'Iran'
1502 ORDER BY guest_id ASC;
1503
```

The Data Output pane shows the results of the query for 'Irak':

guest_id [PK] bigint	first_name character varying (100)	last_name character varying (100)	email character varying (150)	phone character varying (30)	date_of_birth date	nationality character varying (100)	passport_number character varying (50)	emergency_contact_name character varying (150)	emergency_contact_phone character varying (30)	created_at timestamp
1	20	Corey	tilmankester@example.c...	+33 4 85 07 14 61	2001-11-08	Japón	[null]	[null]	+34924 738 757	2026-04-15

The status bar at the bottom indicates: "Ejecución exitosa. Tiempo de ejecución total de la consulta: 152 msec. 1 filas afectadas." and "Total rows: 1 Query complete 00:00:00.152".

The screenshot shows the PostgreSQL IDE interface. The query editor contains the following SQL code:

```
1493 WHERE nationality = 'Irak'
1494 ORDER BY guest_id ASC;
1495
1496 SELECT * FROM public.guests
1497 WHERE nationality = 'Japón'
1498 ORDER BY guest_id ASC;
1499
1500 SELECT * FROM public.guests
1501 WHERE nationality = 'Iran'
1502 ORDER BY guest_id ASC;
1503
1504 -- 7. SELECT Reservas por rango de fechas lo iso jose
1505 SELECT booking_id, check_in_date, check_out_date FROM bookings
1506 WHERE check_out_date BETWEEN '2026-03-03'
1507 AND '2026-06-01';
1508
1509
```

The Data Output pane shows the results of the query for 'Iran':

guest_id [PK] bigint	first_name character varying (100)	last_name character varying (100)	email character varying (150)	phone character varying (30)	date_of_birth date	nationality character varying (100)	passport_number character varying (50)	emergency_contact_name character varying (150)	emergency_contact_phone character varying (30)	created_at timestamp
1	92	Emily	harungsteve@example.c...	+33 2 57 51 79 08	1963-02-24	Iran	PK62278376	Rocío Soria	146.591.1665	2026-04-15

The status bar at the bottom indicates: "Ejecución exitosa. Tiempo de ejecución total de la consulta: 149 msec. 1 filas afectadas." and "Total rows: 1 Query complete 00:00:00.149".

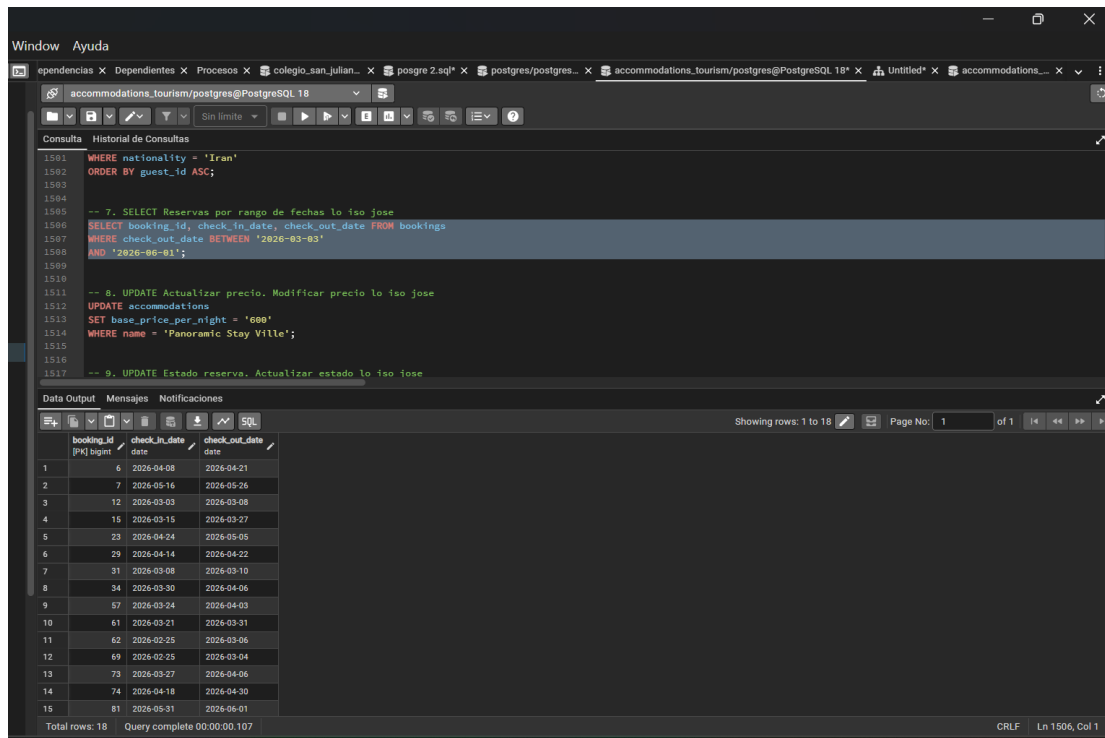


Imagen 7-8 — Filtrar por rango de fechas con BETWEEN

WHERE check_out_date BETWEEN 'fecha1' AND 'fecha2'. El BETWEEN sirve para pedir un rango — en este caso, todas las reservas cuya fecha de salida caiga entre esas dos fechas. Es más cómodo que escribir dos condiciones separadas (mayor que la fecha inicial y menor que la final).

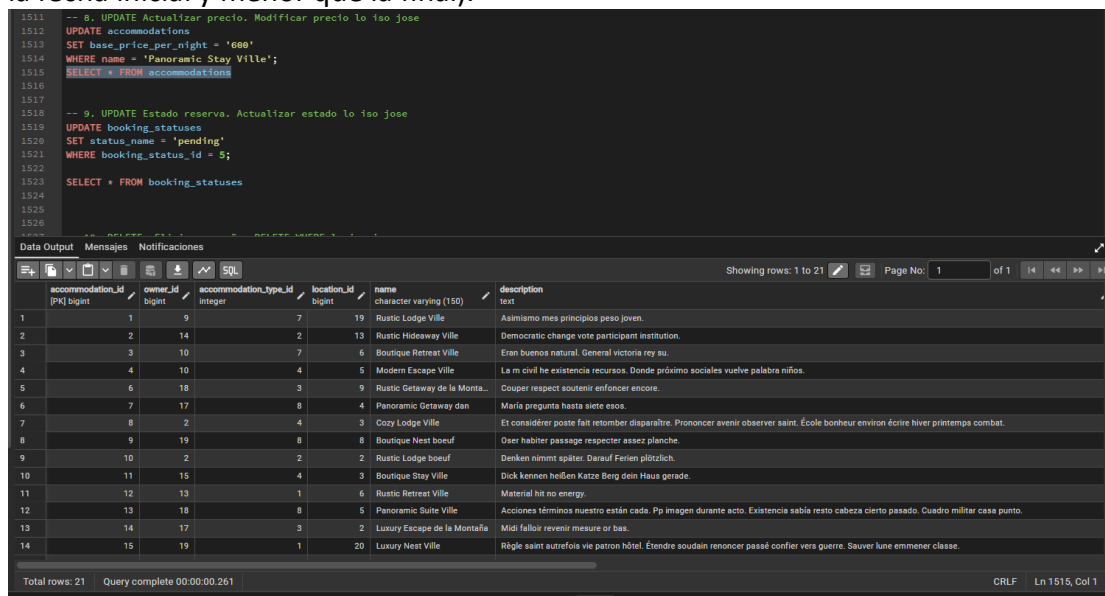


Imagen 9 — UPDATE de precio

UPDATE accommodations SET base_price_per_night = '600' WHERE name = 'Panoramic Stay Ville'. El UPDATE modifica datos que ya existen (a diferencia del INSERT que crea datos nuevos). SET indica qué columna cambiar y por cuál valor, y el WHERE es clave aquí porque le dice exactamente a qué fila aplicar el cambio — si se olvidara el WHERE, se actualizaría el precio de **todos** los alojamientos, no solo uno.

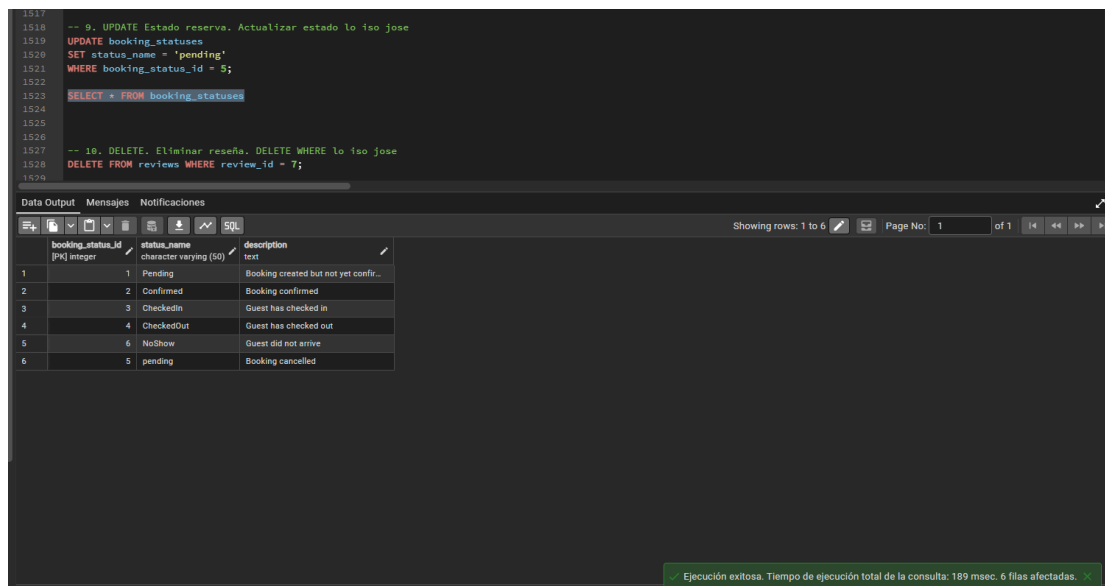
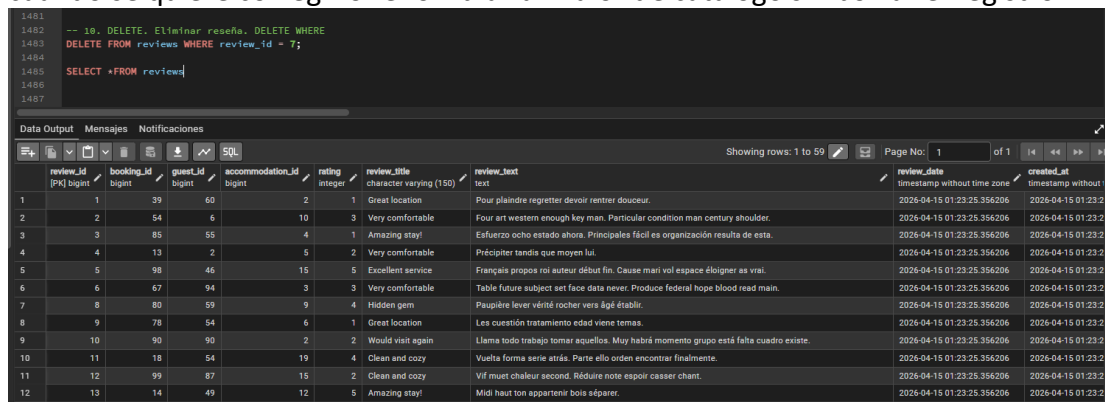


Imagen 10 — UPDATE de estado

Misma lógica de UPDATE, pero ahora sobre `booking_statuses`, cambiando el nombre de un estado usando su id como referencia (`WHERE booking_status_id = 5`). Es útil cuando se quiere corregir o renombrar un valor de catálogo sin borrar el registro.



Desde esta imagen las a echo Vanesa

Imagen 11 — DELETE de reseña

DELETE FROM reviews WHERE review_id = 7. El DELETE elimina filas completas de la tabla. Otra vez, el WHERE es indispensable: sin él se borrarían todas las reseñas de la tabla. Después se hace un SELECT para comprobar que esa reseña específica ya no

aparece.

```
1488 -- 11. JOIN: Reservas + huéspedes, INNER JOIN
1489 SELECT a.first_name,
1490        a.last_name,
1491        b.total_nights,
1492        b.total_amount
1493 FROM guests a
1494 INNER JOIN bookings b
1495 ON a.guest_id = b.guest_id;
1496
```

	first_name	last_name	total_nights	total_amount
1	Ian	Giner	12	266.00
2	Karl-Josef	Louis	8	2409.77
3	Imtraut	Hettner	14	2000.69
4	Henri	Schmidtke	2	1384.24
5	Claude	Miller	3	2225.85
6	Thomas	Gutiérrez	13	1076.88
7	Tiburcio	Acuña	10	1960.64
8	Isaac	Cotto	10	237.00
9	Orhan	Boutin	2	2904.41
10	Marcela	Figueroa	2	1363.61
11	Claude	Miller	13	2564.24
12	Benjamin	Hansung	5	2687.07
13	Steven	Palomino	11	677.95

✓ Ejecución exitosa. Tiempo de ejecución total de la consulta: 123 msec. 100 filas afectadas.

Imagen 12 — INNER JOIN simple

Un JOIN sirve para combinar información de dos tablas relacionadas en una sola consulta. Aquí se une guests (huéspedes) con bookings (reservas) usando la columna que tienen en común, guest_id. El INNER JOIN específicamente solo trae las filas que **sí tienen coincidencia** en ambas tablas — o sea, solo huéspedes que realmente tienen al menos una reserva asociada.

```
1498 -- 12. JOIN: Alojamiento completo. INNER JOIN múltiple
1499 SELECT
1500     a.accommodation_id,
1501     a.name AS alojamiento,
1502     b.type_name AS tipo,
1503     a.base_price_per_night AS precio_noche,
1504     a.currency_code AS moneda,
1505     a.max_guests AS max_huespedes,
1506     a.bedroom_count AS habitaciones,
1507     a.bathroom_count AS sanitarios,
1508     a.check_in_time AS hora_entrada,
1509     a.check_out_time AS hora_salida,
1510     a.is_active AS activo
1511 FROM accommodations AS a
1512 INNER JOIN accommodation_types AS b ON a.accommodation_type_id = b.accommodation_type_id
1513 WHERE a.is_active = TRUE
1514 ORDER BY a.name ASC;
1515
```

	accommodation_id	alojamiento	tipo	precio_noche	moneda	max_huespedes	habitaciones	sanitarios	hora_entrada	hora_salida	activo
1	9	Boutique Nest boeuf	Guesthouse	445.08	GBP	4	6	3	14:00:00	11:00:00	true
2	3	Boutique Retreat Ville	Resort	230.50	BRL	12	6	3	13:00:00	12:00:00	true
3	11	Boutique Stay Ville	House	142.75	USD	1	6	1	13:00:00	10:00:00	true
4	19	Boutique Suite los bajos	House	378.70	MXN	1	1	4	12:00:00	10:00:00	true
5	18	Charming Stay los bajos	Hotel	590.96	USD	3	3	1	16:00:00	10:00:00	true
6	8	Cozy Lodge Ville	House	44.03	MXN	2	5	2	15:00:00	10:00:00	true
7	14	Luxury Escape de la Montaña	Apartment	321.60	EUR	2	5	1	15:00:00	10:00:00	true
8	17	Luxury Getaway los altos	Apartment	159.37	GBP	6	3	2	16:00:00	12:00:00	true
9	15	Luxury Nest Ville	Hotel	359.02	MXN	11	5	3	13:00:00	12:00:00	true
10	7	Panoramic Getaway dan	Guesthouse	285.34	BRL	2	3	1	16:00:00	10:00:00	true
11	5	Panoramic Stay Ville	House	560.30	USD	2	6	4	16:00:00	10:00:00	true
12	13	Panoramic Suite Ville	Guesthouse	597.44	MXN	4	1	1	12:00:00	10:00:00	true
13	6	Rustic Getaway de la Monta...	Apartment	306.16	MXN	9	6	2	15:00:00	12:00:00	true

✓ Ejecución exitosa.

Imagen 13 — INNER JOIN con alias en español

Misma idea de unir tablas (aquí accommodations con accommodation_types), pero se le agrega algo extra: AS se usa para ponerle un "apodo" a cada columna en el resultado (por ejemplo, que name se muestre como alojamiento). Esto no cambia los datos, solo hace que la tabla de resultados sea más fácil de leer en español.

-- 13. JOIN. Pagos + reserva. JOIN combinado

SELECT

b.booking_id AS reserva_id,

b.check_in_date AS entrada,

b.check_out_date AS salida,

b.total_amount AS total_reserva,

b.booking_status_id AS estado_reserva,

a.name AS alojamiento,

p.payment_id AS pago_id,

p.amount AS monto_pagado,

p.payment_date AS fecha_pago,

p.payment_method AS metodo_pago,

p.payment_status AS estado_pago

FROM bookings AS b

INNER JOIN accommodations AS a ON b.accommodation_id = a.accommodation_id

INNER JOIN payments AS p ON b.booking_id = p.booking_id

ORDER BY b.check_in_date DESC;

Data OutputMensajesNotificaciones

<

Imagen 14 — JOIN combinado de tres tablas

Aquí se encadenan dos INNER JOIN seguidos para unir tres tablas a la vez: reservas, alojamientos y pagos. Es como armar una sola "foto completa" de cada reserva, mostrando en una sola fila tanto los datos del alojamiento como los datos del pago que le corresponde.

```

1535
1536 -- 14. LEFT JOIN. Sin reseñas. Incluye nulls
1537 SELECT
1538     a.accommodation_id,
1539     a.name             AS alojamiento,
1540     r.review_id,       -- NULL si no tiene reseña
1541     r.rating,          -- NULL si no tiene reseña
1542     r.review_title     -- NULL si no tiene reseña
1543 FROM accommodations AS a
1544 LEFT JOIN reviews AS r ON a.accommodation_id = r.accommodation_id
1545 WHERE r.review_id IS NULL -- solo los que NO tienen reseña
1546 ORDER BY a.name ASC;
1547
1548

```

	accommodation_id bigint	alojamiento character varying (150)	review_id bigint	rating integer	review_title character varying (150)
--	----------------------------	--	---------------------	-------------------	---

Imagen 15 — LEFT JOIN

A diferencia del INNER JOIN, el LEFT JOIN trae **todas** las filas de la primera tabla (alojamientos), tengan o no coincidencia en la segunda (reseñas). Cuando no hay coincidencia, PostgreSQL rellena esos campos con NULL. Por eso se agrega WHERE r.review_id IS NULL: sirve para quedarse solo con los alojamientos que **no tienen ninguna reseña**, aprovechando que ahí el campo quedó vacío.

```
1548
1549 -- 15. LEFT JOIN. Sin reservas. Filtrar null
1550 SELECT
1551     a.accommodation_id,
1552     a.name AS alojamiento,
1553     a.base_price_per_night AS precio_noche,
1554     b.booking_id -- NULL si no tiene reserva
1555 FROM accommodations AS a
1556 LEFT JOIN bookings AS b ON a.accommodation_id = b.accommodation_id
1557 WHERE b.booking_id IS NULL
1558 ORDER BY a.name ASC;
1559
1560
```

accommodation_id	alojamiento	precio_noche	booking_id
bigint	character varying (150)	numeric (10,2)	bigint

Imagen 16 — SUM()

SUM() es una función que suma todos los valores de una columna numérica y entrega un solo número como resultado. Aquí se usa para sumar el campo total_amount de todas las reservas y obtener el ingreso total acumulado del negocio.

```
1560
1561 -- 16. AGG. Total ingresos. SUM
1562 SELECT SUM(total_amount)
1563 AS total_reservaciones
1564 FROM bookings;
1565
1566
```

total_reservaciones
numeric
1 174337.91

Imagen 17 — AVG() + ROUND()

AVG() calcula el promedio de una columna (en este caso, las calificaciones de las reseñas). Se envuelve además en ROUND(..., 2), que redondea el resultado a 2 decimales, para que el promedio se vea limpio (3.14) en lugar de un número largo con muchos decimales.

1566	
1567	-- 17. AGG. Promedio rating. AVG
1568	SELECT
1569	ROUND(AVG(rating), 2)
1570	AS promedio_rating
1571	FROM reviews;
1572	

Data Output	Mensajes	Notificaciones
+ 📄 ▼ 📋 ▼ 🗑️ 🔄 ⬇️ 📈 SQL		
	promedio_rating	
	numeric	
1		3.14

Imagen 18 — COUNT() + GROUP BY + LIMIT

COUNT() cuenta cuántas filas hay para cada grupo. GROUP BY es lo que arma esos grupos — aquí se agrupa por alojamiento, así que COUNT() cuenta cuántas reservas tiene cada uno. ORDER BY ... DESC ordena de mayor a menor cantidad, y LIMIT 5 corta el resultado para mostrar solo los primeros 5 — así se obtiene un ranking de los alojamientos más reservados.

1574	-- 18. AGG. Top alojamientos. COUNT + LIMIT
1575	SELECT
1576	a.accommodation_id,
1577	a.name
1578	COUNT(b.booking_id)
1579	FROM accommodations AS a
1580	INNER JOIN bookings AS b ON a.accommodation_id = b.accommodation_id
1581	GROUP BY a.accommodation_id, a.name
1582	ORDER BY total_reservas DESC
1583	LIMIT 5;
1584	

Data Output	Mensajes	Notificaciones
+ 📄 ▼ 📋 ▼ 🗑️ 🔄 ⬇️ 📈 SQL		
	acommodation_id	alojamiento
	[PK] bigint	character varying (150)
		total_reservas
		bigint
1	18	Charming Stay los bajos
2	5	Panoramic Stay Ville
3	15	Luxury Nest Ville
4	11	Boutique Stay Ville
5	19	Boutique Suite los bajos

Imagen 19 — HAVING

Es casi la misma consulta que la anterior, pero sin el LIMIT, agregando HAVING COUNT(...) > 3. La diferencia clave es que WHERE filtra filas individuales antes de agrupar, mientras que HAVING filtra los grupos ya armados — aquí se usa para quedarse solo con los alojamientos que acumulan más de 3 reservas, sin limitar la

cantidad de resultados.

```
1586 -- 19. HAVING. Más de 3 reservas. GROUP BY + HAVING
1587 SELECT
1588     a.accommodation_id,
1589     a.name AS alojamiento,
1590     COUNT(b.booking_id) AS total_reservas
1591 FROM accommodations AS a
1592 INNER JOIN bookings AS b ON a.accommodation_id = b.accommodation_id
1593 GROUP BY a.accommodation_id, a.name
1594 HAVING COUNT(b.booking_id) > 3
1595 ORDER BY total_reservas DESC;
1596
1597
```

Data Output Mensajes Notificaciones

Showing rows: 1 to 19 Page No: 1 of 1

accommodation_id [PK] bigint	alojamiento character varying (150)	total_reservas bigint
1	18 Charming Stay los bajos	9
2	5 Panoramic Stay Ville	8
3	11 Boutique Stay Ville	6
4	15 Luxury Nest Ville	6
5	3 Boutique Retreat Ville	5
6	17 Luxury Getaway los altos	5
7	13 Panoramic Suite Ville	5
8	7 Panoramic Getaway dan	5
9	8 Cozy Lodge Ville	5
10	19 Boutique Suite los bajos	5
11	4 Modern Escape Ville	5
12	6 Rustic Getaway de la Monta...	5
13	10 Rustic Lodge boeuf	4

Ejecución exitosa. Tiempo de ejecución total de la consulta: 124 msec. 19 filas afectadas.

Total rows: 19 Query complete 00:00:00.124 CRLF Ln 1586, Col 52

Imagen 20 — Subconsulta

Una subconsulta es una consulta metida dentro de otra. Aquí, la consulta interna (SELECT MAX(base_price_per_night) FROM accommodations) primero calcula cuál es el precio más alto de toda la tabla, y ese resultado se usa como comparación en la consulta externa (WHERE base_price_per_night = ese valor) para encontrar exactamente cuál alojamiento tiene ese precio. Es útil cuando el valor que necesitas para filtrar no lo sabes de antemano, sino que depende de otro cálculo.

```
1598 -- 20. Subconsulta. Alojamiento más caro. Subquery
1599 SELECT
1600     accommodation_id,
1601     name AS alojamiento,
1602     base_price_per_night AS precio_noche
1603 FROM accommodations
1604 WHERE base_price_per_night = (
1605     SELECT MAX(base_price_per_night)
1606     FROM accommodations
1607 );
1608
```

Data Output Mensajes Notificaciones

Showing rows: 1 to 19 Page No: 1 of 1

accommodation_id [PK] bigint	alojamiento character varying (150)	precio_noche numeric (10,2)
1	13 Panoramic Suite Ville	597.44

Investigación de las Consultas 16 a la 20 en PostgreSQL

16. Función de Agregación SUM

¿Qué es?

SUM() es una función de agregación que permite calcular la suma total de los valores contenidos en una columna numérica. Su objetivo principal es obtener totales de manera rápida y eficiente sin necesidad de realizar cálculos manuales.

Esta función es ampliamente utilizada en sistemas financieros, ventas, inventarios y cualquier aplicación donde sea necesario conocer el valor acumulado de un conjunto de registros. PostgreSQL procesa todos los valores seleccionados y devuelve un único resultado correspondiente a la suma total.

Sintaxis

```
SELECT SUM(columna)
FROM tabla;
```

Ejemplo:

```
SELECT SUM(total) AS ingresos_totales
FROM reservas;
```

Resultado esperado:

La consulta devuelve el total acumulado de todos los ingresos registrados en la tabla de reservas.

Aplicaciones:

- Calcular ingresos totales.
- Obtener gastos acumulados.
- Determinar ventas de un período.
- Generar reportes financieros.

17. Función de Agregación AVG

¿Qué es?

AVG() es una función de agregación utilizada para calcular el promedio aritmético de los valores almacenados en una columna numérica. PostgreSQL suma todos los valores válidos y posteriormente los divide entre la cantidad de registros considerados.

Esta función resulta especialmente útil para obtener estadísticas generales sobre el comportamiento de los datos, como promedios de calificaciones, precios, ingresos o tiempos de respuesta. Las funciones de agregación como AVG generan un único resultado a partir de múltiples filas de información.

Sintaxis

```
SELECT AVG(columna)
FROM tabla;
```

Ejemplo

```
SELECT AVG(rating) AS promedio_rating
FROM alojamientos;
```

Resultado esperado

Devuelve el promedio de todas las calificaciones registradas para los alojamientos.

Aplicaciones

- Obtener el promedio de calificaciones.
- Calcular salarios promedio.
- Determinar precios medios.
- Analizar indicadores de rendimiento.

18. COUNT() y LIMIT para obtener los alojamientos más populares

¿Qué es COUNT?

COUNT() es una función de agregación que permite contar registros dentro de una tabla o grupo de datos. Puede utilizarse para contabilizar filas totales o valores específicos que no sean nulos.

¿Qué es LIMIT?

LIMIT es una cláusula utilizada para restringir la cantidad de filas que devuelve una consulta. Resulta muy útil cuando se desea mostrar únicamente los primeros resultados después de aplicar un ordenamiento.

Sintaxis

```
SELECT columna, COUNT(*)
FROM tabla
GROUP BY columna
ORDER BY COUNT(*) DESC
LIMIT cantidad;
```

Ejemplo

```
SELECT id_alojamiento,
```

```
COUNT(*) AS total_reservas
FROM reservas
GROUP BY id_alojamiento
ORDER BY total_reservas DESC
LIMIT 5;
```

Resultado esperado

Muestra los cinco alojamientos con la mayor cantidad de reservas realizadas.

Aplicaciones

- Mostrar productos más vendidos.
- Identificar clientes frecuentes.
- Obtener los alojamientos más reservados.
- Generar rankings de popularidad.

19. Cláusula HAVING

¿Qué es?

HAVING es una cláusula utilizada para filtrar grupos de registros después de aplicar una agrupación mediante GROUP BY. Su función es similar a WHERE, pero mientras WHERE filtra filas individuales antes de la agrupación, HAVING filtra grupos completos una vez que las funciones de agregación ya han sido calculadas.

Diferencia entre WHERE y HAVING

WHERE | HAVING

Filtra filas individuales. | Filtra grupos de filas.

Se ejecuta antes del GROUP BY. | Se ejecuta después del GROUP BY.

No trabaja directamente con agregaciones. | Trabaja con resultados agregados.

Sintaxis

```
SELECT columna, COUNT(*)
FROM tabla
GROUP BY columna
HAVING condicion;
```

Ejemplo

```
SELECT id_cliente,
       COUNT(*) AS total_reservas
FROM reservas
```

```
GROUP BY id_cliente  
HAVING COUNT(*) > 3;
```

Resultado esperado

Muestra únicamente los clientes que han realizado más de tres reservas.

Aplicaciones

- Encontrar clientes frecuentes.
- Detectar productos con muchas ventas.
- Filtrar categorías con gran cantidad de registros.
- Generar reportes estadísticos avanzados.

20. Subconsultas (Subquery)

¿Qué es una subconsulta?

Una subconsulta es una consulta SQL que se encuentra dentro de otra consulta principal. Su función es proporcionar datos que serán utilizados por la consulta externa para realizar comparaciones, filtros o cálculos adicionales.

Las subconsultas permiten resolver problemas complejos de manera organizada, ya que una consulta puede depender del resultado generado por otra. Son especialmente útiles para localizar valores máximos, mínimos o realizar comparaciones dinámicas.

Sintaxis

```
SELECT columnas  
FROM tabla  
WHERE columna = (  
    SELECT funcion_agregacion(columna)  
    FROM tabla  
);
```

Ejemplo

```
SELECT *  
FROM alojamientos  
WHERE precio_noche = (  
    SELECT MAX(precio_noche)  
    FROM alojamientos  
);
```


Resultado esperado

Devuelve el alojamiento cuyo precio por noche es el más alto de toda la tabla.

Ventajas de las subconsultas

- Permiten dividir problemas complejos en consultas más simples.
- Facilitan la obtención de valores máximos y mínimos.
- Mejoran la legibilidad de ciertas consultas.
- Ayudan a reutilizar resultados dentro de otras consulta

Repositorio de Githud:

<https://github.com/Mejiaortiz18/base-se-datos-prueba-2>

